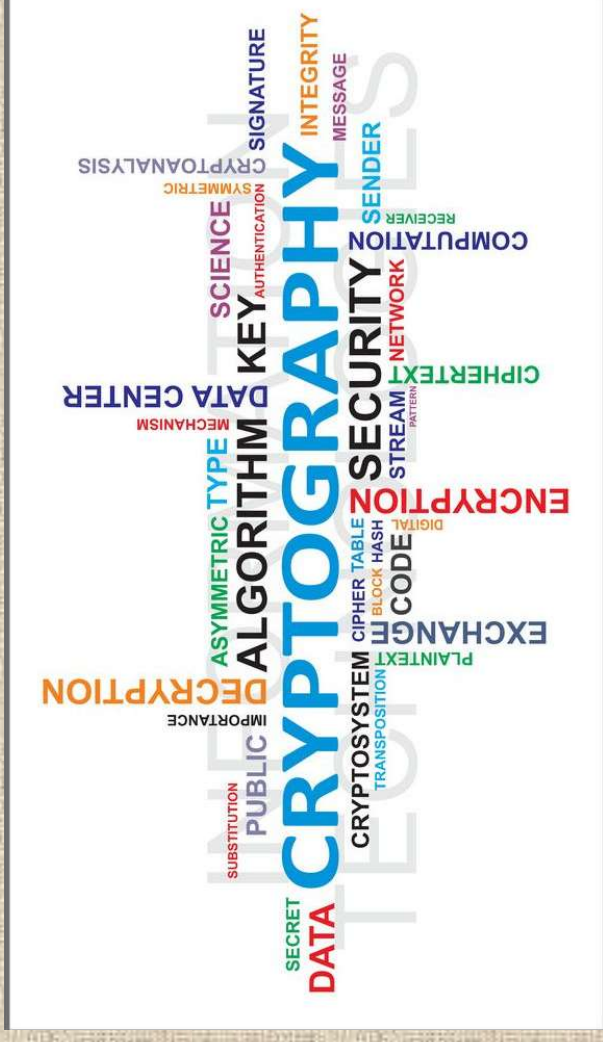


Authentication and Hashing



Authentication

- **Authentication** is the process by which it can be identified that the user, which wants to access the network resources or system, valid or not by asking some credentials such as username and password.
- Authentication assures secure systems, secure processes and enterprise information security.

Authentication Types

- There are several authentication types.
- **Single-factor authentication (SFA):** For purposes of user identity, users are typically identified with a user ID, and authentication occurs when the user provides credentials such as a password that matches their user ID. The practice of requiring a user ID and password is known as *single-factor authentication (SFA)*.
- **Two-factor authentication (2FA):** This type of authentication adds an extra layer of protection to the process by requiring users to provide a second authentication factor in addition to the password. 2FA systems often require the user to enter a verification code received via text message on a preregistered mobile phone or mobile device, or a code generated by an authentication application.
- **Multifactor authentication (MFA):** When three or more identity verification factors are used for authentication -- for example, a user ID and password, biometric signature and perhaps a personal question the user must answer -- it is called *multifactor authentication (MFA)*.

Authentication Requirements

- Data is prone to various attacks. One of these attacks includes message authentication. This threat arises when the user does not have any information about the originator of the message. authentication is required due to the following attacks :

1. **Disclosure:** Release of message contents to any person or process not possessing the appropriate cryptographic key.
2. **Traffic analysis:** Discovery of the pattern of traffic between parties. In a connection oriented application, the frequency and duration of connections could be determined. In either a connection-oriented or connectionless environment, the number and length of messages between parties could be determined.
3. **Masquerade:** Insertion of messages into the network from a fraudulent source. This includes the creation of messages by an opponent that are supposed to come from an authorized entity. Also included are fraudulent acknowledgments of message receipt or non receipt by someone other than the message recipient.

Authentication Requirements

4. **Content Modification:** Changes to the contents of a message, including insertion, deletion, transposition, or modification.
5. **Sequence modification:** Any modification to a sequence of messages between parties, including insertion, deletion, and reordering.
6. **Timing modification:** Delay or replay of messages. In a connection-orientated application, an entire session or sequence of messages could be a replay of some previous valid session, or individual messages in the sequence could be delayed or replayed.
7. **Repudiation:** Denial of receipt of message by destination or denial of transmission of message by source.

Authentication Functions

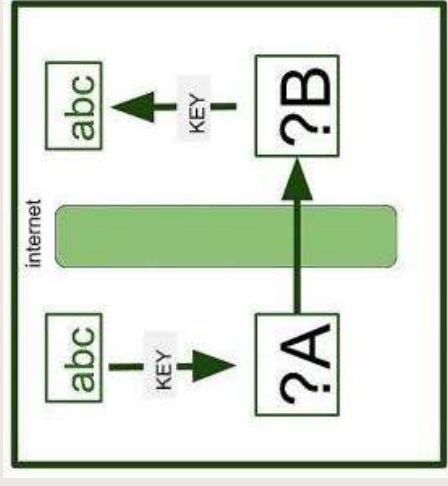
- The types of functions that may be used to produce an authenticator:
1. **Message Encryption:** The ciphertext of the entire message serves as its authenticator.
 2. **Message Authentication Code (MAC):** A public function of the message and a secret key that produces a fixed length value that serves as the authenticator.
 3. **Hash Functions:** A public function that maps a message of any length into a fixed length hash value, which serves as the authenticator.

Message Authentication Code

- **Message Authentication Code (MAC)**, also referred to as a tag, is used to authenticate the origin and nature of a message.
- MACs use authentication cryptography to verify the legitimacy of data sent through a network or transferred from one person to another.
- **In other words**, MAC ensures that the message is coming from the correct sender, has not been changed, and that the data transferred over a network or stored in or outside a system is legitimate and does not contain harmful code.
- MACs can be stored on a hardware security module, a device used to manage sensitive digital keys.
- **Where do we need these codes?**

Where do we need these codes?

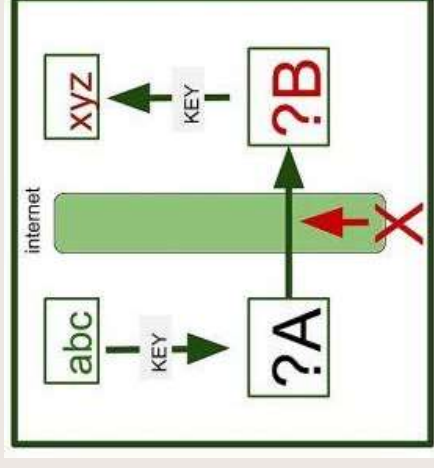
- Suppose User **A** send message to user **B** with message – ‘abc’. **A** encrypts the message using **Shared – Key Cryptosystem** for encrypting the message.
- **A** sends the key to **B** using a source key.
- Key exchange is based on different protocols such as **Public – Key Cryptosystem**. **B** uses the key to decrypt the Ciphertext and obtains the message.



- **All this does not happen so easily. There lies a problem.**

Where do we need these codes?

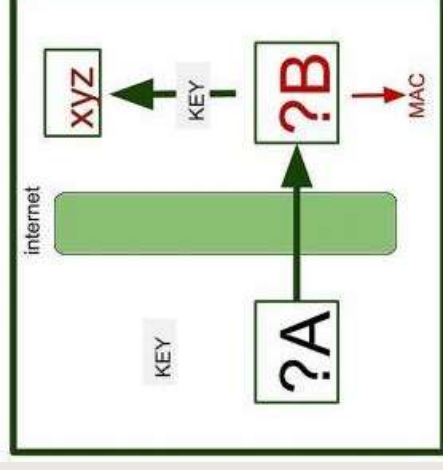
- If a malicious user **X** has falsified the ciphertext during the transmission. Then, in that case, **B** has no way to realize that it has been falsified. When **B** decrypts the message, it will get the wrong message.



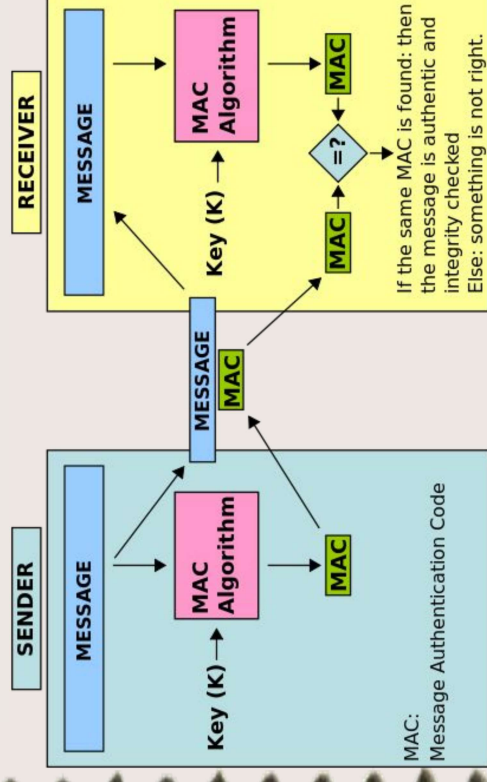
- Here we need to detect the falsification in the message **B** has got.

Where do we need these codes?

- Here **A** will create a key (used to create Message Authentication Code) and sends the key to **B**.
- **A** will create a value using either using plaintext or Ciphertext (if confidentiality needed) and key and the value is obtained. This value Created by
 - **Ciphertext (or plaintext) + Key = Message Authentication Code.**
- **B** has to check whether the ciphertext/plaintext is falsified or not using Message Authentication Code.
- Now **B** can clearly know that whether the ciphertext is falsified or not.



How Does a Message Authentication Code Work?



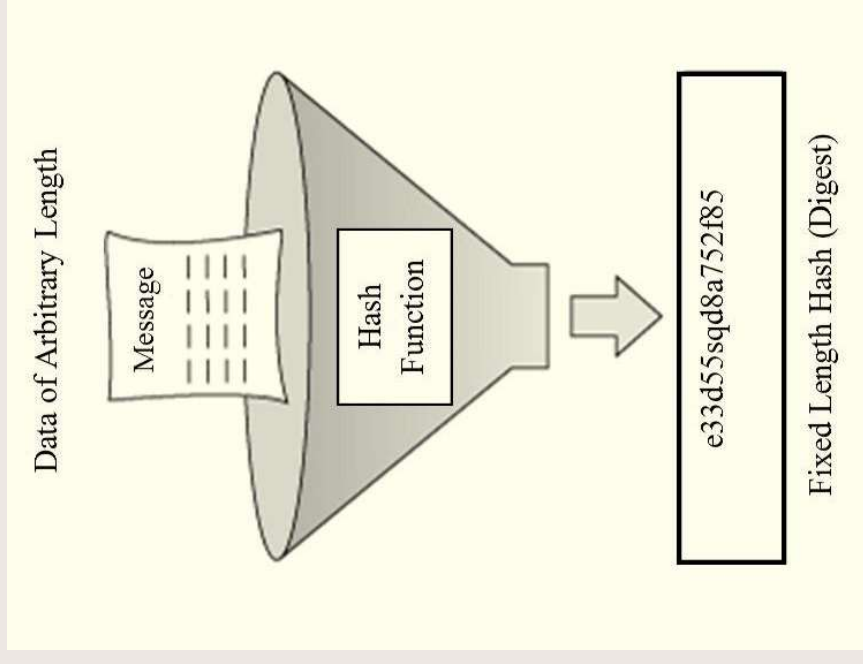
- The sender uses some publicly known MAC algorithm, inputs the message and the secret key K and produces a MAC value.
- MAC function also compresses an arbitrary long input into a fixed length output.
- The sender forwards the message along with the MAC. If confidentiality is required then the message needs encryption.
- On receipt of the message and the MAC, the receiver feeds the received message and the shared secret key K into the MAC algorithm and re-computes the MAC value.
- The receiver now checks equality of freshly computed MAC with the MAC received from the sender. If they match, then the receiver accepts the message and assures himself that the message has been sent by the intended sender.
- If the computed MAC does not match the MAC sent by the sender, the receiver cannot determine whether it is the message that has been altered or it is the origin that has been falsified. **As a bottom-line, a receiver safely assumes that the message is not the genuine.**

What Are the Benefits of Message Authentication Codes?

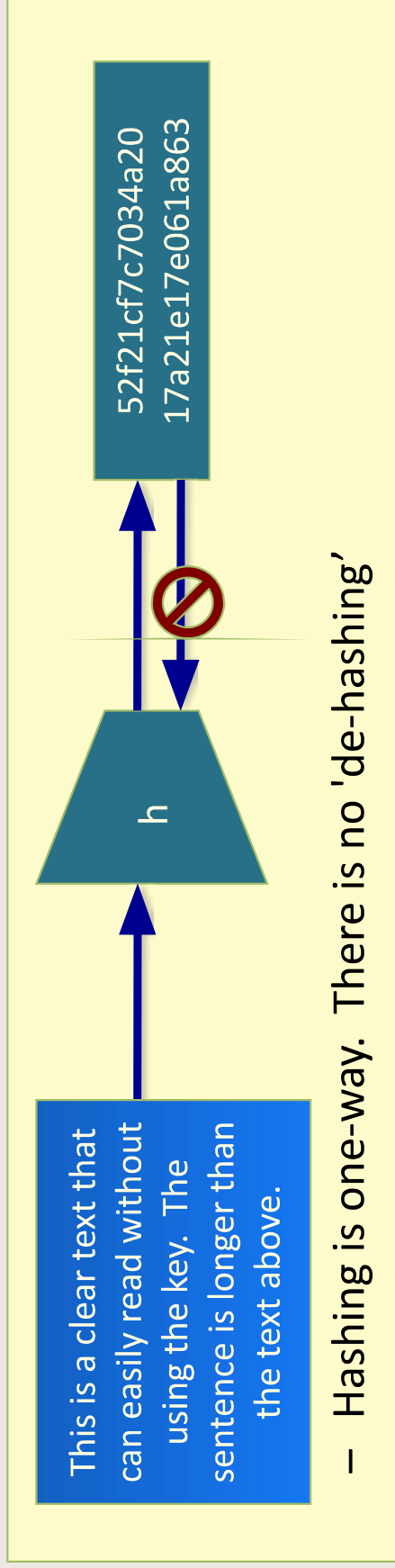
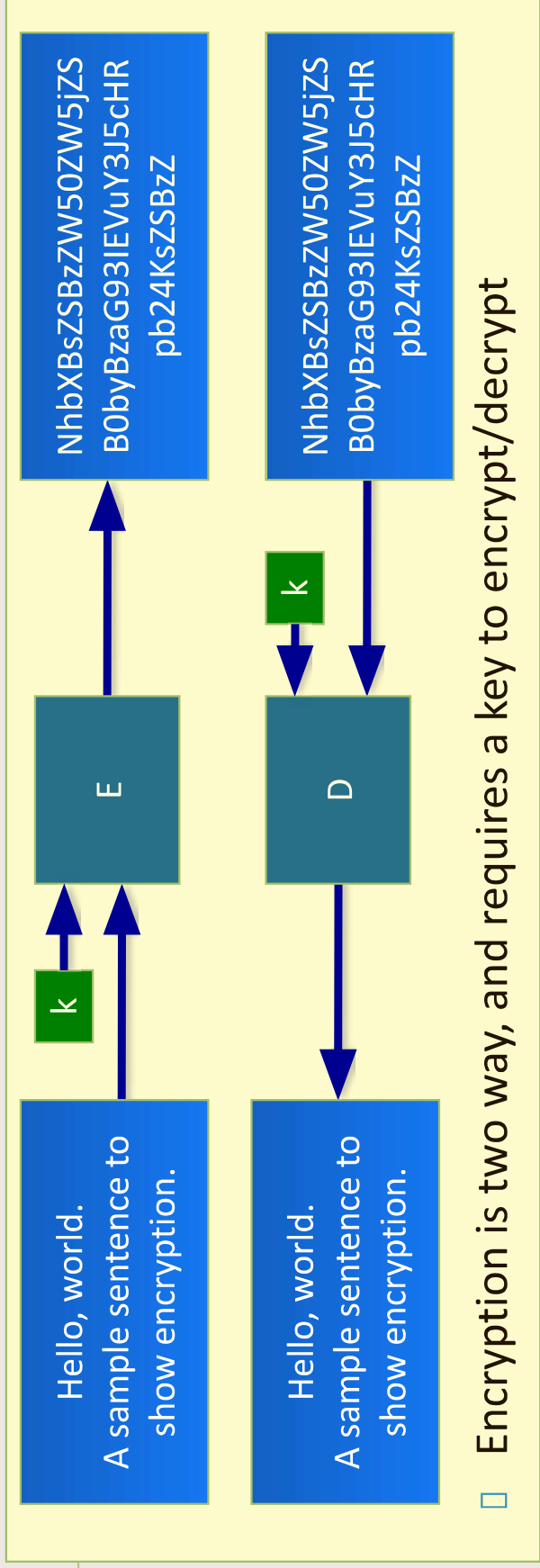
- **1. Protects Data Integrity**
 - With MACs, you can make sure that unauthorized code, such as executable codes used by viruses, has not been put into your system. This is useful when trying to combat *viruses* and other *malware*.
- **2. Detects Changes in the Message Content**
 - You can use an application to generate a MAC based on data that has been sent to you or provided via a storage device. After the application generates a MAC, it can be compared to the original one to detect changes to the data.

Hashing

- A hash function is a mathematical function that converts a numerical input value into another compressed numerical value.
- The input to the hash function is of arbitrary length but output is always of fixed length.
- Values returned by a hash function are called message digest or simply hash values.



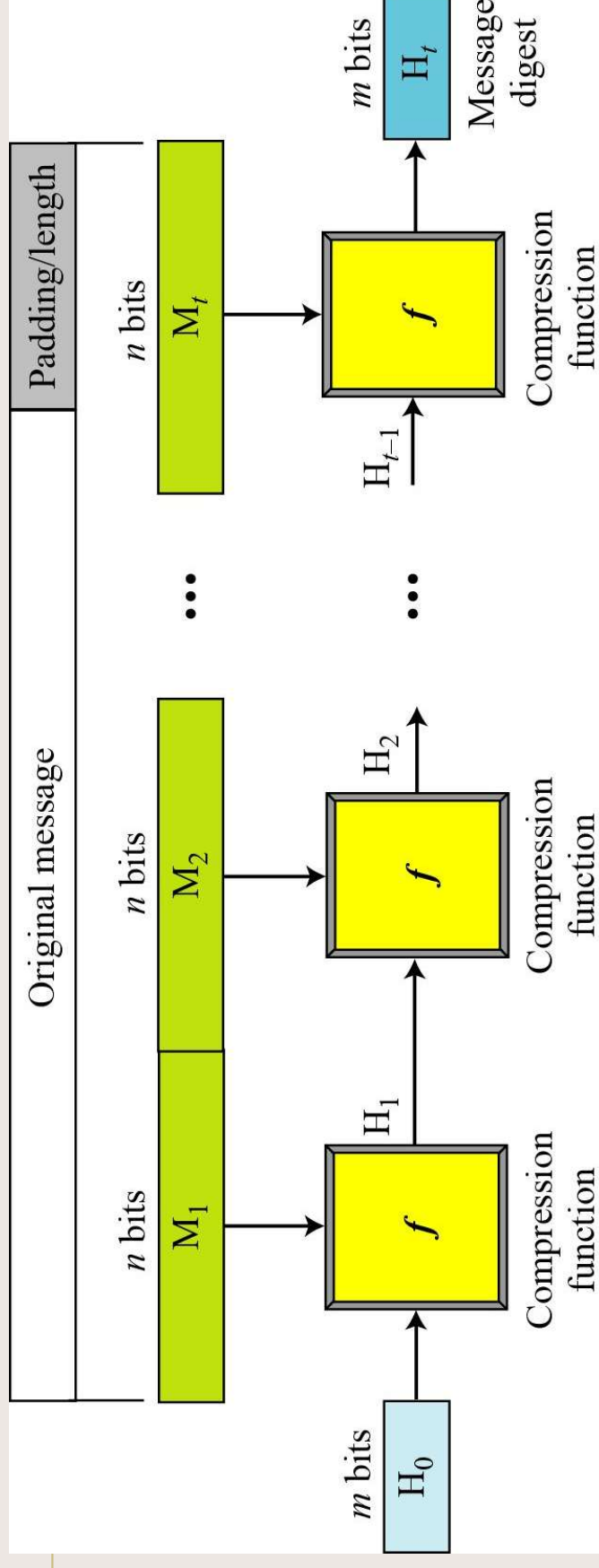
Hashing V.S. Encryption



Features of Hash Functions

- **Fixed Length Output (Hash Value)**
 - Hash function converts data of arbitrary length to a fixed length.
 - Popular hash functions generate values between 160 and 512 bits.
- **Efficiency of Operation**
 - Computationally hash functions are much faster than a symmetric encryption.
- **Pre-Image Resistance**
 - This property means that it should be computationally hard to reverse a hash function.
 - In other words, if a hash function h produced a hash value z , then it should be a difficult process to find any input value x that hashes to z .
- **Second Pre-Image Resistance**
 - This property means given an input and its hash, it should be hard to find a different input with the same hash.
- **Collision Resistance**
 - This property means it should be hard to find two different inputs of any length that result in the same hash. This property is also referred to as collision free hash function.

Basics of Design of Hashing Algorithms



- Merkle-Damgård Scheme is Well-known method to build cryptographic hash function
- A message of arbitrary length is broken into blocks
 - length depends on the compression function f
 - padding the size of the message into a multiple of the block size.
 - sequentially process blocks, taking as input the result of the hash so far and the current message block, with the final fixed length output

Design of Hashing Algorithms

- Damgard and Merkle greatly influenced cryptographic hash function design by defining a hash function in terms of what is called a *compression function*.
- A compression function takes a fixed-length input and returns a shorter, fixed-length output.
- Since, the hash value of first message block becomes an input to the second hash operation, output of which alters the result of the third operation, and so on. This effect, known as an **avalanche effect** of hashing.
- Avalanche effect results in substantially different hash values for two messages that differ by even a single bit of data.

Popular Hash Algorithms

- **Message digest (MD)**
 - The MD family comprises of hash functions MD2, MD4, MD5 and MD6. It is a 128-bit hash function.
- **SHA (Secure Hash Algorithm)**
 - Family: SHA-0, SHA-1, and SHA-2
 - SHA-2: SHA-224, SHA-256, SHA-384, SHA-512
 - SHA-3: New standard in competition
 - **Secure hashing algorithm 256 (SHA256)**
 - This is a cryptographic hash algorithm that creates a 256-bit (32 bytes) string value as the hash value. SHA256 often checks the data integrity for hash authentication and digital signatures.

Popular Hash Algorithms

- **RIPEMD**

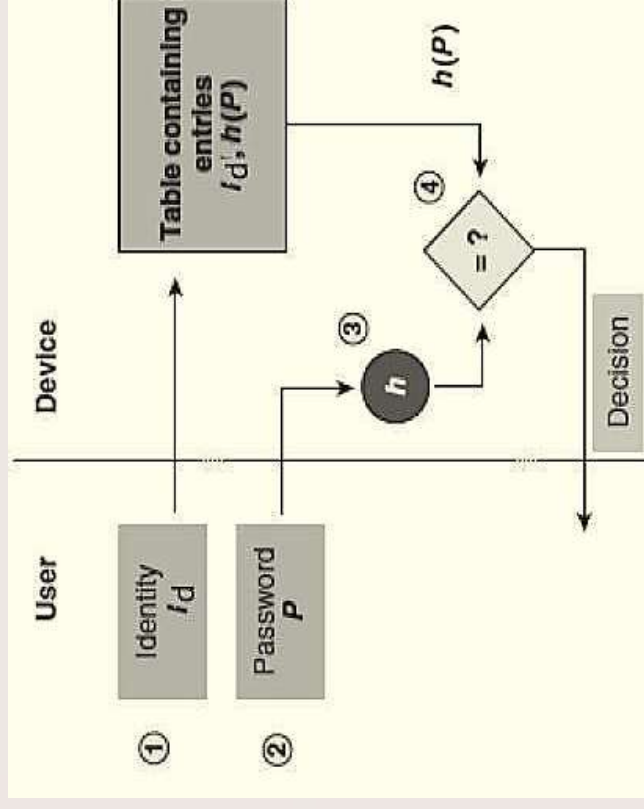
- The RIPEMD is an acronym for RACE Integrity Primitives Evaluation Message Digest. The set includes RIPEMD, RIPEMD-128, and RIPEMD-160. There also exist 256, and 320-bit versions of this algorithm.

- **Whirlpool**

- It is derived from the modified version of Advanced Encryption Standard (AES). One of the designer was Vincent Rijmen, a co-creator of the AES.
- Three versions of Whirlpool have been released; namely WHIRLPOOL-0, WHIRLPOOL-T, and WHIRLPOOL.

Applications of Hash Functions

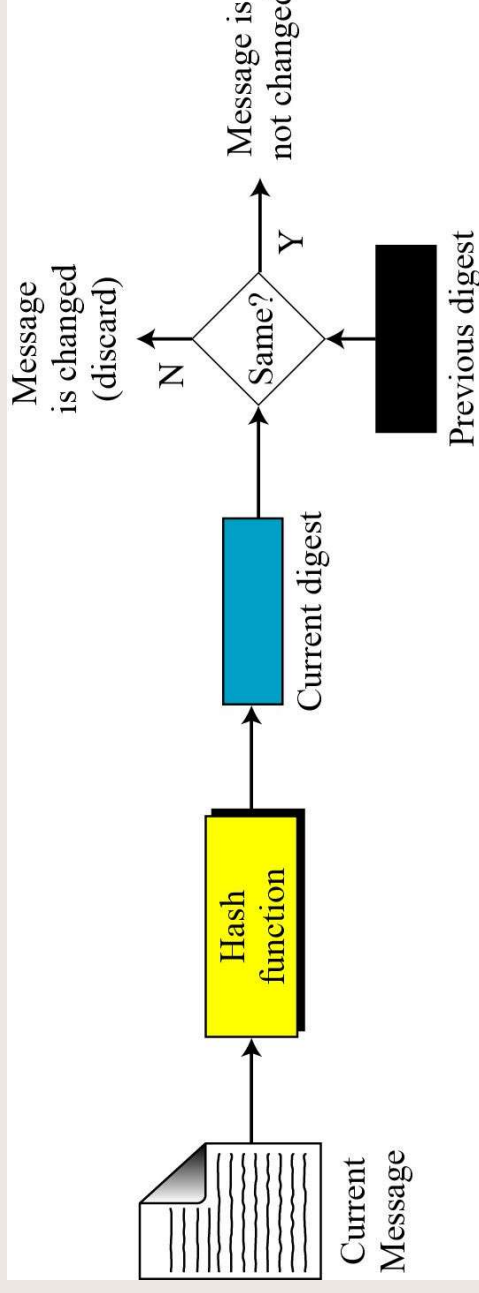
- There are two direct applications of hash function based on its cryptographic properties.
- **Password Storage**
 - Instead of storing password in clear, mostly all logon processes store the hash values of passwords in the file.
 - The Password file consists of a table of pairs which are in the form (user id, $h(P)$).
 - The process of logon is depicted in the following illustration –



An intruder can only see the hashes of passwords, even if he accessed the password. He can neither logon using hash nor can he derive the password from hash value since hash function possesses the property of pre-image resistance.

Applications of Hash Functions

- **Data integrity check:** It is a most common application of the hash functions. It is used to generate the checksums on data files. This application provides assurance to the user about correctness of the data.



- **to create a one-way password file**
 - store hash of password not actual password
- **for intrusion detection and virus detection**
 - keep & check hash of files on system

Digital Signature

- A digital signature is a mathematical technique used to validate the authenticity and integrity of a message, software, or digital document.
- Digital Signature needs a public key system. The sender uses a private key to sign a document and the verifier uses the public key to verify the document.
- A valid digital signature helps the receiver to know the message comes from the authentic sender and is not altered in between.

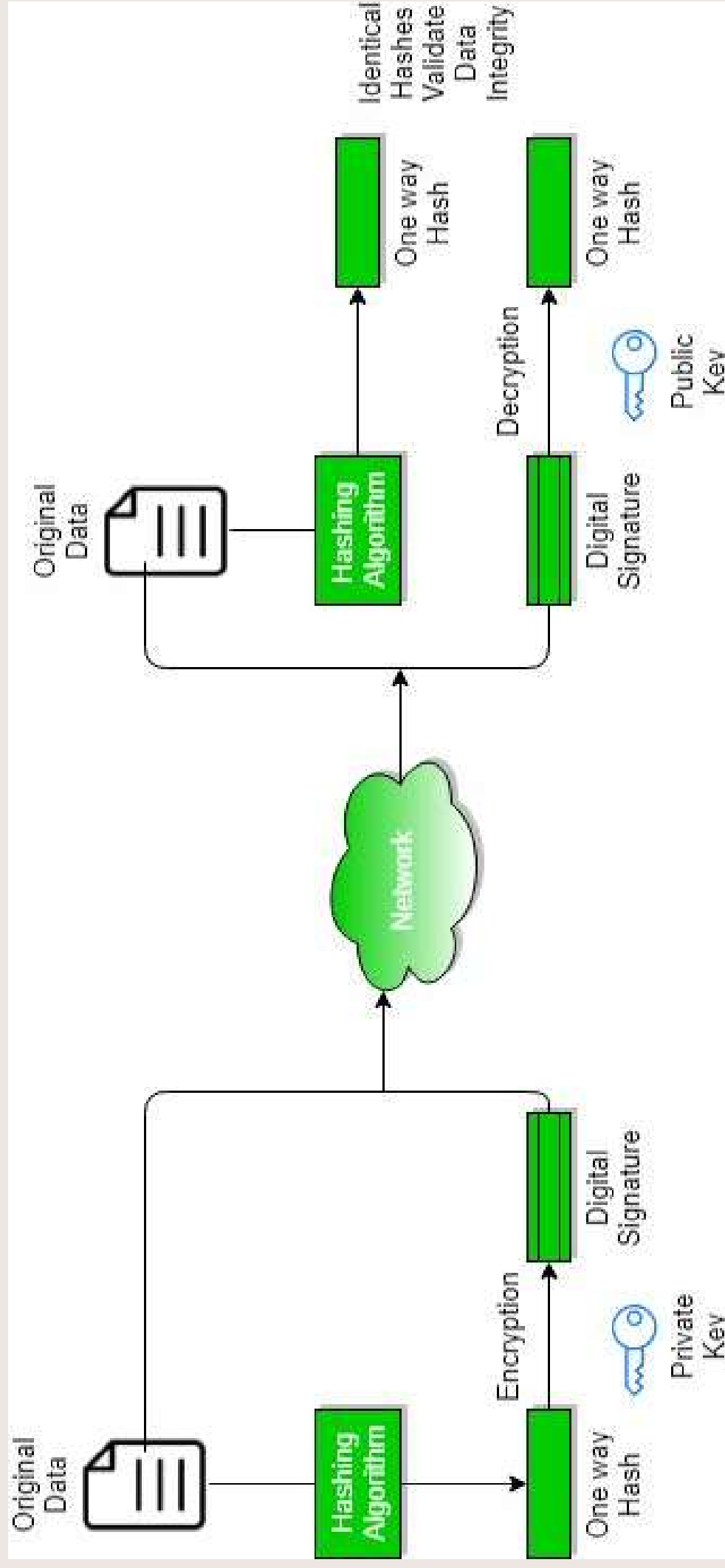
Importance of Digital Signature

- Apart from ability to provide non-repudiation of message, the digital signature also provides message authentication and data integrity.
 - **Message authentication** – When the verifier validates the digital signature using public key of a sender, he is assured that signature has been created only by sender who possess the corresponding secret private key and no one else.
 - **Data Integrity** – In case an attacker has access to the data and modifies it, the digital signature verification at receiver end fails. The hash of modified data and the output provided by the verification algorithm will not match. Hence, receiver can safely deny the message assuming that data integrity has been breached.
 - **Non-repudiation** – Since it is assumed that only the signer has the knowledge of the signature key, he can only create unique signature on a given data. Thus the receiver can present data and the digital signature to a third party as evidence if any dispute arises in the future.

Algorithms in Digital Signature

- A digital signature consists of three algorithms:
- **1. Key generation algorithm:** The key generation algorithm selects private key randomly from a set of possible private keys. This algorithm provides the private key and its corresponding public key.
- **2. Signing algorithm:** A signing algorithm produces a signature for the document. Signing Algorithms make one-way hashes of the data that has to be signed. Then they encrypt the hash value using the signature key. The encrypted hash along with the other information is the Digital Signature.
- **3. Signature verifying algorithm:** A signature verifying algorithm either accepts or rejects the document's authenticity.

How to Create a Digital Signature?



How to Create a Digital Signature?

1. Message digest is computed by applying hash function on the message and then message digest is encrypted using private key of sender to form the digital signature. (**digital signature = encryption (private key of sender, message digest)** and **message digest = message digest algorithm(message)**).
2. Digital signature is then transmitted with the message.(**message + digital signature**) is transmitted)
3. Receiver decrypts the digital signature using the public key of sender.(This assures authenticity, as only sender has his private key so only sender can encrypt using his private key which can thus be decrypted by sender's public key).
4. The receiver now has the message digest.
5. The receiver can compute the message digest from the message (actual message is sent with the digital signature).
6. The message digest computed by receiver and the message digest (got by decryption on digital signature) need to be same for ensuring integrity.

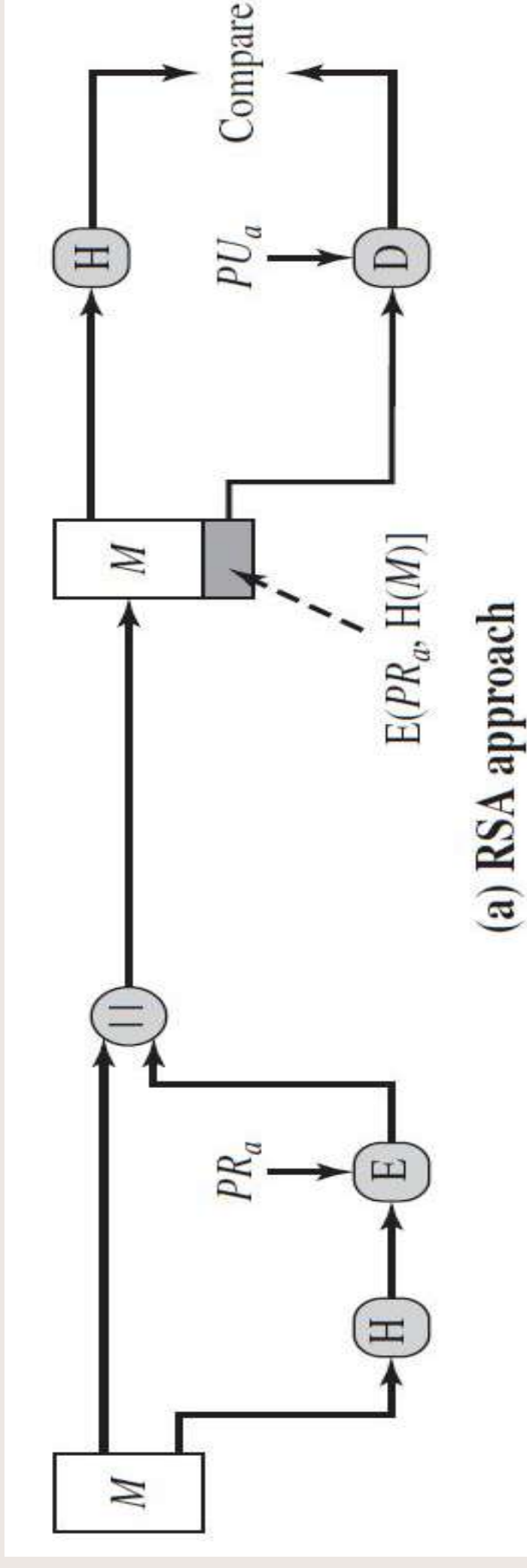
Digital Signature Standard

Digital Signature Standard (DSS) is a Federal Information Processing Standard(FIPS) which defines algorithms that are used to generate digital signatures with the help of Secure Hash Algorithm(SHA) for the authentication of electronic documents.

DSS only provides us with the digital signature function and not with any encryption or key exchanging strategies.

Digital Signature Standard:

The RSA Approach for Digital Signatures



- Working of it discussed in previous slides.

The DSA (Digital Signature Algorithm)

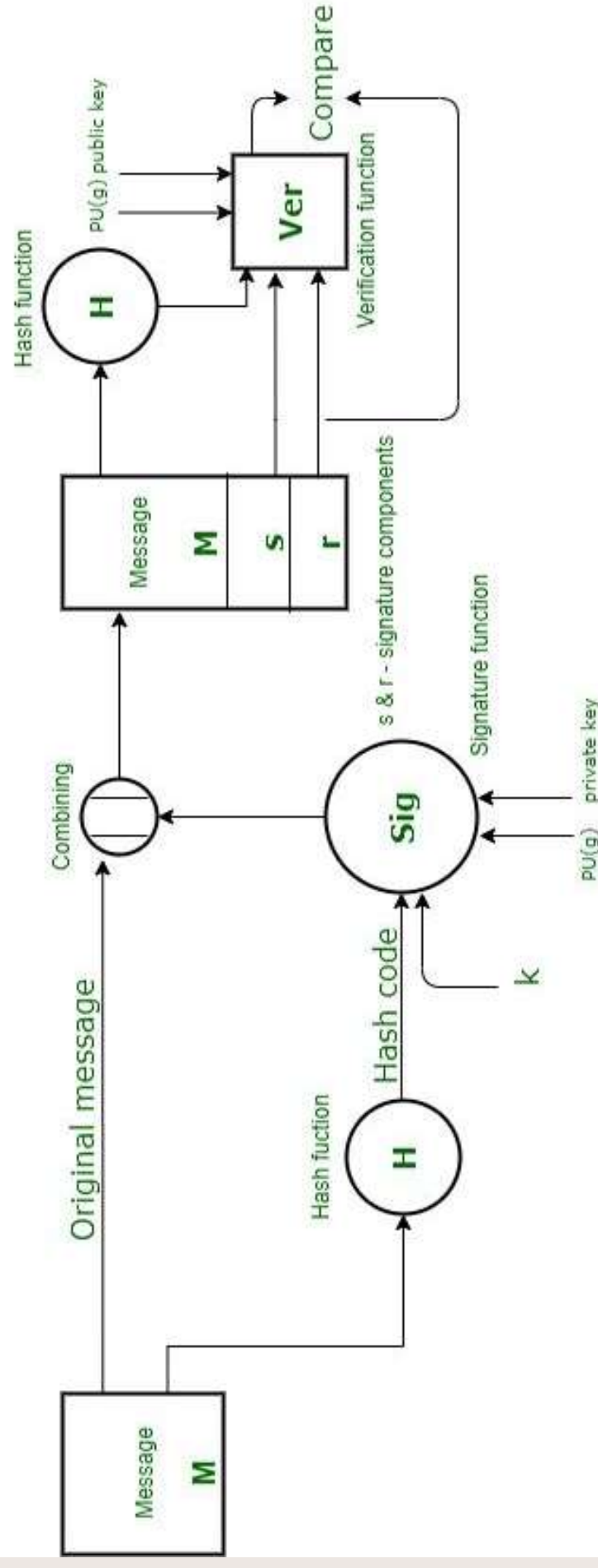
Approach

- DSS Approach, a hash code is generated out of the message and following inputs are given to the signature function –
 - The hash code.
 - The random number 'k' generated for that particular signature.
 - The private key of the sender i.e., $PR(a)$.
 - A global public key (which is a set of parameters for the communicating principles) i.e., $PU(g)$.
- See the figure in the next side

The DSA Approach

SENDER A

RECEIVER B



The DSA Approach

- We can consider this set to constitute a global public key (PUG).
- The result is a signature consisting of two components, labeled s and r .
- Therefore, the original message concatenated with the signature is sent to the receiver.
- **Receiver Side :** At the receiver end, verification of the sender is done. The hash code of the sent message is generated. There is a verification function which takes the following inputs –
 - The hash code generated by the receiver.
 - Signature components ' s ' and ' r '.
 - Public key of the sender.
 - Global public key.

The DSA

- The output of the verification function is a value that is equal to the signature component r if the signature is valid.
- The signature function is such that only the sender, with knowledge of the private key, could have produced the valid signature.